Fundamentals of System Security
(COMSM0122)

Workshop 1: Designing a System that Scales to Millions of Users

Prerequisite:
(1) A code editor such as Visual Studio Code
(2) Node.js (version 18 or newer)
(3) Basic familiarity with JavaScript
(4) A GitHub account

Use cases

Knowledge of specific cloud architectures such as AWS or Azure is not required for this exercise. However, many of the principles you will learn and tasks performed can be applied more generally in any cloud computing ecosystem**.** In this exercise, you will design a system that incrementally scales from tens to *hundreds* to thousands and, subsequently, millions of user requests. At each point, you will ensure that your system architecture is robust and can withstand denial-of-service attacks. You will evaluate the effectiveness of your architecture at each level using a design configuration and load-balancing testbed.

We'll scope the problem to handle only the following use cases:

1. The user makes a read or write request to the system, with each request carrying a data payload.
2. The server provides the user with a session service that processes and stores the request payload in a database.
3. The system needs to evolve from serving a small amount of users to millions of users
4. Discuss general scaling patterns as you evolve the design architecture to handle an increasing number of users and requests.
5. The system is cost-effective, highly available and can withstand denial-of-service attacks.

Constraints and assumptions

a) User traffic is not evenly distributed. The size of each request's payload therefore varies.

b) You have the option of selecting from one of three load-balancing algorithms. These are Round Robin, Least Connections and Consistent Hashing.

c) You may choose to scale vertically, horizontally, or both. However, you are limited by the following:
   - The number of user request sessions that a server can handle at any point in time is limited.
   - Each server can have multiple replicas up to a limit of 100
   - Server performance is not evenly distributed, with each varying by the processing time required to satisfy a request.

d) All servers satisfy a request by writing its payload to a datastore and responding to the user on a successful write. To achieve this function, servers are connected to one or more databases. The capacity of databases is not evenly distributed.

Setup: Load-Balancing Testbed

- Check that NodeJS is installed by running the command node on the terminal. You can download and install the latest version from https://nodejs.org/en
- Copy and unzip the folder LoadBalancing.zip from Blackboard into your workspace directory on VSCode.

The testbed depends on "find-free-ports" package on npm for NodeJS HTTP server configuration. To install this package run npm install find-free-ports

Objective

You will use the above use case, constraints and assumptions in the following hands-on lab. The objective is to work in groups of 2 to transform the requirements above into a highly-available, resilient and secured system design architecture. You will achieve this objective by addressing the tasks listed below.

Task A

1. Create a high-level design diagram of an architecture that scales for tens of users.

2. Generate an architectural configuration file containing a config object with the following variable settings:

```
config = {
   LoadBalancingAlgorithm: <value>,
   NoServers: <value>,
   Replicas: <value>,
   MaxNoSessionsPerServer: <value>,
   MinNoSessionsPerServer: <value>,
   MaxProcessingTime: <value>, //seconds
   MinProcessingTime: <value>, //seconds
   NoDatastores: <value>,
   MaxDatastoreSpace: <value>, //GB
   MinDatastoreSpace: <value>, //MB
   MaxSizeRequestPayload: <value>, //MB
   MinSizeRequestPayload: <value>, //MB
}
```

3. To do an initial manual setup of your configuration, run the command: node MainTerminal.js on your terminal. This will enable you to manually set your design configuration and to execute the test bed.
   To test the configuration, open a second terminal and run the command: curl http://localhost:8080/<request> - where <request> is the payload. The request is successful if the response is "Session ACCESS GRANTED" and fails if the response is "Session ACCESS DENIED". A session is denied when either due to lack of space in the datastore or an unavailable server to handle the session.

4. Create a new JavaScript file “RequestSimulation.js” and implement a function that curls tens of server requests every 500, 450, … 30, 20, 15, 10, 5, 1 milliseconds, as well as record the success and failure rates.

5. Evaluate the effectiveness of your configuration in Task (2) by discussing the failure/success rate results in Task (4)

6. Generate five more configurations in task (2) and repeat tasks (4) and (5). Compare your configurations by discussing the ability of each to satisfy highly-available, resilient requirements and the ability to withstand denial-of-service attacks.

7. Repeat all Tasks above for (a) *hundreds*, (b) thousands and, (c) millions of user requests.

Task B

1. **Implement and evaluate fault tolerance**
   Extend the testbed to detect and respond to failures. Add health checks and a way to simulate a backend server becoming unavailable while requests are being processed. Optionally include datastore failure as a second scenario. Implement a recovery strategy, such as removing unhealthy servers from rotation and restoring them when they recover.
   Evaluate: Run the same workload with and without the fault-tolerance features. Record request success rate, failed or lost requests, and recovery time. Explain any trade-offs, such as extra health-check traffic or requests delayed during recovery.
   Deliver: Updated architecture diagram, implementation, test results, and a short analysis of failure behaviour.

2. **Design and test defences against abusive traffic**
   Create a controlled workload that mixes legitimate requests with simulated abusive traffic, such as bursts of requests or requests with oversized payloads. Develop a threat model identifying likely attack paths and system resources at risk. Implement appropriate defences - for example, request-size limits, rate limiting, and timeouts—and test their effect on both abusive and legitimate requests.
   Evaluate: Compare results with and without the defences. Report the rate of legitimate requests served, abusive requests rejected, and any impact on latency or resource use. Run the simulation locally against the lab testbed; do not send attack traffic to external systems.
   Deliver: Threat model, implemented defences, test results, and a brief assessment of the security and usability trade-offs.

Submission

Submit your implementation of both tasks as a zip file on Blackboard.

Notes:

1. Submission points will only be open on the day of the workshop.

2. You must be physically present in the lab to submit (except permitted otherwise)
3. You will not gain marks for submitting.
4. Submissions help us differentiate learning and engagement levels when marking both group and individual coursework.

   For all submissions before group assessment, **you will lose 1 mark for each missed submission up to a maximum of 5 marks as an indication of lack of engagement in learning**.
   For all submissions after group assessment and before individual assessment, **you will lose 1 mark for each missed submission up to a maximum of 5 marks as an indication of lack of engagement in learning**.

5. Finally, push your LoadBalancing project folder to GitHub. Ensure that the project is associated with appropriate documentation and a README.md description. Tag your project appropriately, including the workshop date and name. Finally, invite the GitHub user systemsecuritybcsg to your pushed project. This GitHub handle is managed by the unit director.

.